

INTELIGENCIA ARTIFICIAL:

PRÁCTICA I

Daranuta Zop Cristian - 55261830Y // Esquinas Fernández Guillermo - 49750576N

1. Árbol de Decisión

1.1 Diseño y Implementación

El árbol fue implementado de forma iterativa con tal de evitar la recursión profunda y problemas de *stack overflow*. Se utiliza un enfoque basado en pilas para construir el árbol.

Decisiones clave:

- Compatibilidad con atributos numéricos y categóricos
- Uso de la impureza de Gini o la Entropía como criterios de división
- Introducción de un parámetro beta para penalizar las divisiones débiles
- Poda posterior opcional basada en el umbral de aumento de la impureza

Cada nodo guarda:

- El índice de características (`column`)
- El valor dividido (`value`)
- Nodos secundarios (`true_branch`, `false_branch`)
- Distribución de clases para nodos hoja

El recorrido del árbol durante la predicción sigue la misma lógica de división utilizada durante el entrenamiento

1.2 Criterios de división

At each node, all possible splits are evaluated:

- **Numerical features:** threshold-based split using unique values.
- **Categorical features:** all non-empty, non-complete subsets.

The information gain is computed as:

- $\text{Gain} = \text{Parent impurity} - \text{weighted child impurities} - \text{beta}$

Only splits with positive gain are considered valid.

1.3 Estrategia de poda

After training, the tree is pruned bottom-up. If both children of a node are leaves, they may be merged if the increase in impurity does not exceed the `prune_threshold`.

This strategy reduces overfitting and improves generalization.

1.4 Pruebas en Arboles de Decisión

1.4.1 Efecto de la poda

Prune Threshold	Train acc	Test acc
0.0	100,00%	97.78%
0.05	99.05%	97.78%
0.1	99.05%	97.78%

Observación: Cuanto menor sea el umbral que podamos, mejores resultados nos dan a la hora de entrenar, aunque no afecta a la hora de las pruebas

1.4.2 Effect of Beta Parameter

Beta	Train acc	Test acc
0.0	100,00%	97.78%
0.05	98.1%	97.78%
0.1	95.24%	97.78%

Observación: cuanto mayor sea el beta, peores resultados da a la hora de entrenar el algoritmo

1.4.3 Efecto de combinar Beta y Poda

- Usar solo beta simplifica el árbol durante la construcción
- Usar solo poda elimina ramas innecesarias después del entrenamiento
- Los mejores resultados se dan cuando los dos mecanismos son activados, combinando regularización preventiva y posterior

1.4.4 Comparacion en la impureza

Criterio (beta 0.05 & Prune Threshold 0.1)	Train acc	Test acc
Gini	98.1%	97.78%
Entropy	100,00%	97.78%

Observación: Los dos algoritmos tienen unos resultados parecidos, aunque Gini es más rápido, la Entropy da mejores resultados de entrenamiento

2. Multinomial Naive Bayes

2.1 Diseño y Implementación

La implementación del algoritmo se basa en utilizar atributos de la clase MultinomialNaiveBayesClassifier (vocabulary, class_counts, ...) para almacenar todos los datos del set de entrenamiento para, posteriormente, realizar los cálculos respectivos a las probabilidades sobre a qué clase pertenece el mensaje (ham o spam) que se está analizando según las palabras que contiene.

Para evitar encontrarse con probabilidades nulas, las cuales se dan en caso que la palabra que se esté analizando no aparezca en los sets de entrenamiento, se ha aplicado un Laplace Smoothing. La fórmula resultante es:

- $\text{word_prob} = \frac{x \text{ apariciones en docs de la clase} + \text{assumed_prob}}{\text{total palabras en la clase} + \text{assumed_prob} * \text{total palabras vocabulario}}$

Lo que hace esto es añadir apariciones ficticias en el numerador (según assumed_prob) y se equilibra con el denominador con el total de palabras del vocabulario. Teniendo esto en cuenta, la probabilidad queda correcta y forzamos que no pueda ser de 0.

2.2 Bayesian Experiments

2.2.1 Effect of Laplace Smoothing

Assumed Probability	Train acc	Test acc
0.1	99.56%	98.27%
0.5	99.33%	97.85%
1.0	99.28%	97.97%

Observación: Cuanto menor es el valor de assumed_probability mejores son los resultados dado que esto implica que se reduce el número de apariciones ficticias cuando se realizan los cálculos con Laplace Smoothing.

2.2.2 Comparison with Decision Tree

Test-Ratio	Train acc	Test acc
0.3	99.28%	97.97%
0.6	99.51%	97.31%
0.9	99.64%	96.47%

Observación: en cuanto menor sea el entrenamiento, peor sera los resultados de prueba

3. k-Means Clustering

3.1 Diseño y implementación

Métrica de distancias

La implementación soporta diferentes tipos de distancias, como la *Euclidean* o la *squared Euclidean*.

Inicialización y Reinicios

Los centroides son inicializados en posiciones aleatorias de las k observaciones del conjunto de datos.

Con tal de reducir la sensibilidad a la inicialización, del algoritmo se ejecuta varias veces (*n_restarts*) y se selecciona la configuración con la menor distancia total.

Criterio de convergencia

Cuando el algoritmo en dos iteraciones ve que los centroides quedan en la misma posición. Esto garantiza asignaciones de clusters estables

Clusters vacíos

Si un centroide queda vacío, se reinicia en un punto random del data point, así garantizando siempre k clusters.

3.2 Estudio del Numero de Clusters (k)

k	Suma de distancias	Distancia media	Mayor cluster (%)
2	5719.46	57.77	72.7%
4	4944.18	49.94	68.7%
8	4393.15	44.38	62.6%
16	3673.04	37.10	55.6%
32	3017.66	30.48	30.3%

Observación: Los resultados muestran una creciente distancia entre centroides a medida que aumenta k.

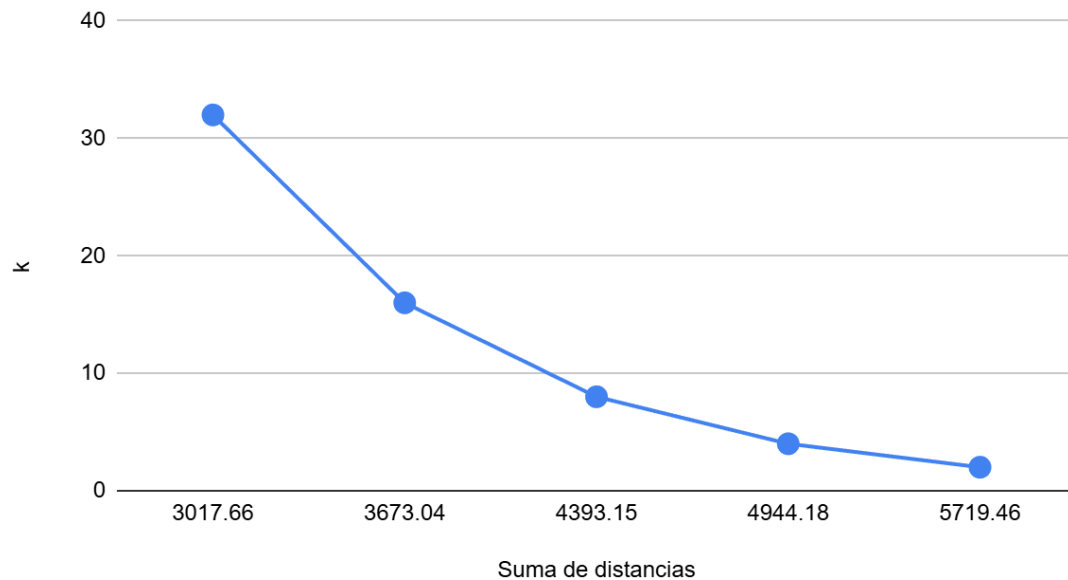
3.3 Elbow Method

Al representar la suma de distancias, en función del número de clusters, se observa una curva con forma de codo.

Dicho codo, aparece aproximadamente a partir de $k = 8$, este punto es a partir del cual, al aumentar el número de clusters, no produce muchas mejoras en la calidad del agrupamiento.

Este valor representa un buen compromiso entre la calidad del clustering y la complejidad del modelo, ya que permite obtener clusters suficientemente compactos sin introducir un número excesivo de centroides.

k frente a Suma de distancias



4. Problemas encontrados

Los principales problemas encontrados en este trabajo fueron:

- Saber cómo construir un árbol de forma iterativa
- Problemas a la hora de visualización de los datos y resultados por terminal
- Problemas en reconocer iteraciones entre clusters

5. Conclusions

- Los árboles de decisión se benefician tanto de la regularización beta como del *post-pruning*
- Naive Bayes destaca en problemas de clasificación de texto de alta dimensión
- El *elbow method* proporciona una heurística eficaz para seleccionar k en k-means

Los algoritmos implementados demuestran un rendimiento sólido y se ajustan a las expectativas teóricas aprendidas en clase